PRODUCT REQUIREMENTS DOCUMENT

Color Analyzer Tool

Pixel-Level Color Distribution Analysis

Document Version	1.0
Status	Draft
Date	June 2025
Author	Product Management
Platform	Google Colab (Python Notebook)
Target Audience	Engineering / Development Team

1. Product Overview

The Color Analyzer Tool is a Python-based image analysis utility that processes any raster image and computes the percentage breakdown of its pixels across 11 standardized color categories. The tool runs as an interactive Google Colab notebook: the user uploads one or more images via a browser upload button, the script analyzes each image, renders an inline dual-chart visualization (pie chart + bar chart), prints a text breakdown to the console, and saves the output charts for download.

This document defines all functional requirements, inputs, algorithm design, output specifications, and edge case handling to a level of detail sufficient for a developer to build the feature without further clarification. All section references to specific thresholds, hue ranges, and implementation choices reflect the accepted v1 codebase.

2. Objectives & Success Criteria

2.1 Objectives

- Accept any number of image uploads through Google Colab's native file upload interface.
- Classify every pixel in each image into exactly one of 11 defined color categories.
- Output the percentage of pixels per category, sorted descending.
- Render an inline dual-chart (pie + bar) visualization per image.
- Save each chart as a PNG to /content/ and offer download.
- Print a formatted text breakdown with an ASCII bar chart to the console.
- Skip unsupported file formats with a warning; never crash on bad input.

2.2 Success Criteria

Criterion	Threshold
Supported image formats	JPEG, PNG, BMP, TIFF, WebP
Color categories	Exactly 11 (see Section 5)
Classification accuracy (vs. human annotation)	Within ±2% per category on reference images
Processing time (10 MP image, after downsampling)	Under 5 seconds on Google Colab free tier
Invalid format handling	Skip with warning; do not raise an unhandled exception
Output chart format	PNG saved to /content/<stem>_color_chart.png
Zero-value categories	Omitted from chart if < 0.1% (threshold for display)

3. Scope

3.1 In Scope

- Google Colab notebook interface with browser-based image upload (google.colab.files.upload()).
- Vectorized pixel-level RGB reading and custom HSV conversion using NumPy.
- Rule-based color classification into 11 categories (priority-override system).
- Dual inline chart: pie chart and bar chart rendered side-by-side via Matplotlib.
- Console text output with ASCII distribution bars per color.
- PNG chart saved to /content/ with auto-download cell.
- Downsampling of large images before analysis.

3.2 Out of Scope

- Command-line interface (CLI) or standalone Python script.
- Web application, REST API, or browser-based GUI outside of Colab.
- Real-time video or camera feed analysis.
- Color naming beyond the 11 defined categories.
- Semantic understanding of image content.
- Cloud storage, database integration, or persistent state between sessions.

---

4. Users & Use Cases

4.1 Primary User

The primary user is a non-technical or semi-technical individual comfortable using Google Colab. No local Python installation, terminal access, or file path knowledge is required. The entire workflow happens through Colab's browser interface.

4.2 Use Cases

ID	Use Case	Description
UC-01	Single image upload	User uploads one image file. Tool analyzes it, prints results, renders chart inline, saves PNG.
UC-02	Multi-image upload	User selects multiple files in the upload dialog. Tool processes each sequentially, one chart per image.
UC-03	Unsupported format upload	User uploads a file with an unsupported extension (e.g. .gif, .pdf). Tool prints a warning and skips it without crashing.
UC-04	No files uploaded	User dismisses the upload dialog without selecting any file. Tool prints a warning and exits the cell cleanly.
UC-05	Download all charts	After analysis, user runs the download cell. Tool calls files.download() for every saved chart PNG.
UC-06	Large image input	User uploads a high-resolution image (≥10 MP). Tool downsamples before analysis; result is indistinguishable from full-resolution processing.

---

5. Color Category Definitions

The tool classifies every pixel into exactly one of 11 categories. These categories are mutually exclusive and collectively exhaustive. The display color used to represent each category in charts is also specified here, as it is hardcoded in the implementation.

#	Category	Display Hex	Plain Description
1	White	#F0EDE8	Very bright, near-neutral pixels (paper, snow, highlights)
2	Black	#1C1C1C	Very dark pixels at or near absolute darkness
3	Gray	#7F8C8D	Low-saturation, mid-brightness pixels (concrete, overcast sky)
4	Red	#C0392B	Chromatic red-hued pixels not dark enough to be Brown
5	Orange	#E67E22	Chromatic orange-hued pixels not dark enough to be Brown
6	Yellow	#F1C40F	Chromatic yellow-hued pixels
7	Green	#27AE60	Chromatic green-hued pixels (includes yellow-green and blue-green)
8	Blue	#2471A3	Chromatic blue-hued pixels (includes cyan and indigo range)
9	Purple	#7D3C98	Chromatic purple/violet-hued pixels
10	Pink	#E91E8C	Chromatic pink/magenta-hued pixels
11	Brown	#7D4E24	Warm-hued (red/orange range) pixels with low brightness — muted, earthy tones

6. Inputs

6.1 Upload Mechanism

Images are provided by the user via Google Colab's built-in upload widget, invoked by:

```
uploaded = files.upload()
```

This renders a 'Choose Files' button inline in the notebook. The user selects one or more files from their local machine. The result is a dictionary mapping filename (str) to raw file bytes (bytes). No file paths, no terminal input, no configuration required.

6.2 Supported Formats

Extension	Format
.jpg / .jpeg	JPEG
.png	PNG
.bmp	BMP
.tiff / .tif	TIFF

Extension	Format
.webp	WebP

Any file with an extension outside this set must be skipped with a printed warning. Extension matching is case-insensitive.

6.3 Image Size Handling (Downsampling)

Before classification, large images are resized to keep total pixel count at or below 250,000. The resize preserves aspect ratio and uses LANCZOS resampling.

Condition	Action
Total pixels ≤ 250,000	Process at original resolution
Total pixels > 250,000	Compute scale = ⌊ √(W×H / 250,000)⌋ , resize to (W//scale, H//scale) using Image.LANCZOS

Rationale: Color distribution is a statistical property. Sampling theory confirms that 250,000 pixels yields a representative estimate with error typically below 0.3% per category. Processing every pixel in a 10 MP image would be ~40× slower with no meaningful accuracy gain.

---

7. Color Classification Algorithm

7.1 Overview

The classification pipeline is fully vectorized using NumPy. It processes all pixels simultaneously in three steps: (1) convert the pixel array from RGB to HSV, (2) assign hue-based chromatic categories, (3) apply a sequence of override rules in priority order. The last override wins.

7.2 RGB to HSV Conversion

A custom vectorized HSV conversion is implemented directly in NumPy (rgb_to_hsv_fast). This avoids per-pixel Python loops and is substantially faster than calling colorsys.rgb_to_hsv on individual pixels. The function accepts an Nx3 uint8 array and returns three float32 arrays:

Output	Range	Description
H (Hue)	0.0 – 360.0 degrees	Chromatic angle; undefined for achromatic pixels (set to 0)
S (Saturation)	0.0 – 1.0	Colorfulness relative to brightness

Output	Range	Description
V (Value)	0.0 – 1.0	Brightness

The conversion formula follows the standard HSV definition: V = max(R,G,B), S = (V − min(R,G,B)) / V, H computed from the dominant channel. An epsilon of 1e-6 guards against division by zero on achromatic pixels.

7.3 Classification Rules (Priority / Override Order)

Rules are applied sequentially to a results array initialized to 'Red'. Each rule overwrites prior assignments for matching pixels. The final state of the array after all rules is the classification output.

Step 1 — Hue-based chromatic assignment (sets baseline for all pixels):

Category	Hue Range (degrees)	Notes
Red (default)	0–14 and 340–360	Array initialized to Red; no explicit rule needed for this range
Orange	15 – 44	
Yellow	45 – 69	
Green	70 – 164	Includes yellow-green and blue-green
Blue	165 – 259	Includes cyan and indigo
Purple	260 – 289	
Pink	290 – 339	

Step 2 — Brown overrides (applied after hue step, overwrites Red and Orange for dark warm pixels):

Override Name	Condition	Overwrites
brown_red	H ≥ 340° AND V ≥ 0.08 AND V < 0.45 AND S ≥ 0.20	Red
brown_orange	H ≥ 10° AND H < 40° AND V ≥ 0.08 AND V < 0.55 AND S ≥ 0.25	Orange

Step 3 — Achromatic overrides (applied last, in this exact order):

Priority	Category	Condition	Notes
1 (first)	Gray	S < 0.15	Catches all low-saturation pixels regardless of hue or brightness
2	White	S < 0.12 AND V > 0.86	Tighter saturation + high brightness; overwrites Gray for bright neutrals
3 (last)	Black	V < 0.14	Absolute darkness wins over everything; applied last so it cannot be overwritten

Important: because rules are applied as array assignments (not early-exit conditionals), later rules overwrite earlier ones. The effective priority from highest to lowest is: Black > White > Gray > Brown > chromatic hues. Developers must preserve this exact application order.

7.4 Percentage Computation

After classification, the percentage for each category is computed as:

```
pct[category] = count(pixels == category) / total_pixels * 100
```

All 11 categories are always computed. A category with 0 pixels yields 0.0%. The percentages across all 11 categories sum to exactly 100%.

7.5 Known Limitations

- HSV thresholds are fixed values. Boundary pixels (e.g. S = 0.14 vs S = 0.16) may look visually identical yet fall into different categories. This is an accepted trade-off.
- Brown has no dedicated hue in color science; it is dark, desaturated orange/red. The Brown rules are the most heuristic part of the algorithm and may misclassify unusual earth tones.
- A future improvement would switch to CIE Lab color space with a nearest-centroid classifier for perceptually uniform classification. Not required for v1.

8. Outputs

8.1 Inline Dual Chart

For each successfully analyzed image, a side-by-side chart is rendered inline in the Colab notebook using Matplotlib. The chart is not interactive; it is a static figure displayed via plt.show().

Property	Specification
Figure size	18 × 7 inches, facecolor #F8F8F8
Figure title	Derived from filename: stem.replace('_',' ').replace('-',' ').title(). Positioned above both subplots.
Layout	1 row, 2 columns (GridSpec), horizontal spacing wspace=0.35
Left panel	Pie chart — wedges colored per COLOR_DISPLAY, white edges, labels show category name + percentage, no autopct
Right panel	Bar chart — bars colored per COLOR_DISPLAY, percentage labels above each bar, rotated x-axis labels (30°), background #EFEFEF
Bar edge color	White bars use #AAAAAA edge; all other bars use their own fill color as edge
Y-axis upper limit (bar)	max(values) × 1.2
Display threshold	Categories with < 0.1% are excluded from both charts (they are included in the text output)
Rendering	plt.show() followed by plt.close(fig) to free memory

8.2 Chart PNG File

Property	Specification
Save path	/content/<stem>_color_chart.png (stem = filename without extension)
DPI	150
Bounding box	bbox_inches='tight'
Background	facecolor='#F8F8F8'
Trigger	Saved automatically during analysis; no user action required

8.3 Console Text Output

For each image, a formatted text block is printed to the Colab console. Format:

```
Analyzing: <filename>
Color       %      Distribution
----------  ------  --------------------------
Blue         27.3%  ████████████
Gray         22.8%  ██████████
...
```

Rules: rows are sorted by percentage descending. Only categories ≥ 0.1% are printed. The ASCII bar is drawn as ▉ characters, one per 2 percentage points (int(val / 2)).

8.4 Download Cell

A separate notebook cell runs after analysis. It scans /content/ for all files matching *_color_chart.png and calls files.download() on each, triggering a browser download. If no charts exist, it prints a prompt to run the upload cell first.

---

9. Error Handling

Scenario	Expected Behavior
No files uploaded (empty dict)	Print: '□ No files uploaded.' and exit the cell cleanly.
File has unsupported extension	Print: '□ Skipping <filename> — unsupported format (<ext>)' and continue to next file.
File bytes cannot be opened as an image (corrupt)	Pillow will raise an exception; this should be caught and logged as a skip with the filename and error message. Do not crash the notebook.
All uploaded files are invalid	After skipping all, no charts are generated. The download cell will print 'No charts found yet.'
/content/ directory not writable	Out of scope for v1 (always writable in Colab). No special handling required.

---

10. Notebook Structure

The implementation is organized as a Google Colab notebook (.ipynb) with the following cell sequence. Each cell is independently runnable, but cells 1–4 must be run before cell 5.

Cell #	Type	Purpose
1	Code	Import all libraries (numpy, PIL, matplotlib, warnings, io, os, glob, IPython.display, google.colab.files). Print confirmation.
2	Code	Define COLOR_DISPLAY dict (11 categories → hex colors) and CATEGORIES list. Print confirmation.
3	Code	Define rgb_to_hsv_fast() — vectorized NumPy HSV conversion.
4	Code	Define classify_pixels() — priority-override classification logic. Define analyze_image_bytes() — opens image, downsamples, calls classifier, returns pct

Cell #	Type	Purpose
		dict. Define show_chart() — renders and saves dual chart. Print 'Ready to analyze images!'
5	Code	Upload cell: calls files.upload(), iterates uploaded dict, calls analyze_image_bytes() and show_chart() per file, prints text breakdown, collects saved chart paths.
6	Code	Download cell: globs /content/*_color_chart.png, calls files.download() for each.

11. Technical Requirements

11.1 Platform

- Google Colab (Python 3 runtime). No local installation required.
- All listed libraries are pre-installed in the default Colab environment. No pip install step is needed.

11.2 Dependencies

Library	Source	Purpose
numpy	Pre-installed (Colab)	Vectorized pixel array operations and HSV conversion
Pillow (PIL)	Pre-installed (Colab)	Image reading, format detection, resizing, pixel access
matplotlib	Pre-installed (Colab)	Pie chart, bar chart, figure layout, PNG export
matplotlib.gridspec	Pre-installed (Colab)	Side-by-side subplot layout
io	Python stdlib	BytesIO wrapper for in-memory image opening from raw bytes
os	Python stdlib	File extension extraction (os.path.splitext)
glob	Python stdlib	Scanning /content/ for chart files in the download cell
warnings	Python stdlib	Suppresses non-critical library warnings (warnings.filterwarnings('ignore'))
IPython.display	Pre-installed (Colab)	display() used for inline rendering confirmation
google.colab.files	Colab-specific	files.upload() and files.download() for browser file transfer

11.3 Performance

- All pixel classification uses vectorized NumPy operations. No Python-level for-loop over individual pixels is permitted (would be 10–100× slower).
- A 10 MP image (downsampled to ≤250,000 pixels) must complete analysis and chart rendering in under 5 seconds on Colab free tier.

---

12. Non-Functional Requirements

Requirement	Specification
Platform	Google Colab only for v1. No local environment support required.
Reproducibility	Running the notebook twice on the same image must produce identical percentage output.
Memory	Each image is processed from in-memory bytes (io.BytesIO); no temp files written to disk except the output chart PNGs.
Clean failure	No cell may raise an unhandled exception on valid or invalid user input. All error paths must be caught and printed as warnings.
Chart display	plt.close(fig) must be called after each plt.show() to free memory when processing many images.
No hardcoded content	No filenames, image paths, or pixel data may be hardcoded. All inputs flow through the upload widget.

---

13. Open Questions & Future Considerations

ID	Item	Status
OQ-01	Should a browser-based GUI (Gradio or Streamlit) be offered as an alternative to the Colab notebook?	Not in scope for v1. Potential Phase 2.
OQ-02	Should Brown be broken into sub-categories (e.g. Dark Brown vs Tan)?	Deferred. 11-category system sufficient for v1.
OQ-03	Should the algorithm be upgraded to CIE Lab + nearest-centroid classification for improved perceptual accuracy?	Noted improvement path. Not required for v1.
OQ-04	Should results be exportable as JSON or CSV for downstream use?	Candidate for v1.1.
OQ-05	Should zero-percentage categories appear in the chart (currently filtered at < 0.1%)?	Open. Current behavior omits them from charts but includes them in text output.

Appendix: Glossary

Term	Definition
HSV	Hue-Saturation-Value: a color model separating chromatic content (hue) from colorfulness (saturation) and brightness (value). More intuitive for color classification than RGB.
RGB	Red-Green-Blue: the standard digital color model where each pixel is described by three 0–255 channel values.
Pixel	The smallest addressable unit of a raster image. Each pixel has one RGB color value.
Downsampling	Reducing an image's total pixel count before processing to improve speed without meaningfully affecting statistical accuracy.
Vectorized operation	A NumPy operation applied simultaneously to an entire array rather than element-by-element in a Python loop. Orders of magnitude faster for large pixel arrays.
CIE Lab	A perceptually uniform color space where equal numerical distances correspond to equal perceived color differences. Noted as a future improvement over HSV.
Google Colab	A hosted Jupyter notebook environment by Google, running Python in the browser with free access to GPUs and pre-installed data science libraries.
LANCZOS	A high-quality image resampling filter used during downscaling. Minimizes aliasing artifacts compared to simpler methods like nearest-neighbor.

End of Document